Exercise 1: Short Read Alignment

76553 - Computational Genomics

Mohammad Awawda

Project link in github
https://github.com/mohammadawawda/Computational-genomics-Aligner.git

Part A: Custom Seed Based Aligner

A.1 Algorithm Description

The aligner uses a seed and extend strategy, which is the standard approach for mapping short reads to a reference genome efficiently. Instead of comparing every read to every possible position in the genome (which would be computationally expensive), we break each read into short fragments called seeds, look up where those seeds appear in the genome using a pre-built hash index, and then verify each candidate location by checking the full read alignment. This dramatically reduces the search space.

The algorithm has two main phases. In the indexing phase, we scan the entire reference genome and record the position of every k-mer (a substring of length k) in a hash table. This only needs to be done once and allows constant time lookup of any k mer. In the alignment phase, for each read, we extract a few distinct seeds, query the hash table for candidate genomic positions, and then do a character by character comparison at each candidate to count mismatches. We keep the position with the fewest mismatches, as long as it falls within our tolerance.

Key Assumptions and Design Choices

- We use k=14 for the seed length. This strikes a balance: smaller k mers produce too many hits per lookup (wasting time on repetitive sequences), while larger k mers are more likely to miss valid alignments when the seed itself contains a mismatch.
- We allow up to 2 mismatches in the full read alignment. Since the reads are short (36-40bp) and come from a single individual mapped back to the reference, most true alignments will have very few mismatches. Two mismatches accounts for both single nucleotide polymorphisms and sequencing errors.
- We extract 3 distinct seeds per read, spaced evenly along the read. Using multiple seeds increases sensitivity: even if one seed falls on a variant or a sequencing error, another seed can still find the correct location.

- We skip seeds that map to more than 500 positions in the genome. These correspond to highly repetitive sequences and are unlikely to produce a unique, meaningful alignment. Skipping them is a crucial efficiency optimization.
- Each read is aligned to both the forward strand and the reverse complement, since DNA is double stranded and the read could have originated from either strand.
- We align each read individually (not as pairs), as stated in the exercise instructions. Case differences in the reference are ignored by converting everything to uppercase.

A.2 Pseudocode

Building the Index

We slide a window of size 14 across the entire genome, one base at a time. At each position, we take the 14 letter substring and store it in a lookup table along with where we found it. If the substring contains any unknown bases (N), we skip it. When we're done, we can instantly look up any 14 letter sequence and get back all the places it appears in the genome.

Reverse Complement

Since DNA is double-stranded, a read might come from either strand. To check the opposite strand, we swap each base with its pair (A with T, C with G) and then reverse the whole sequence.

Counting Mismatches

To check how well a read matches a specific spot in the genome, we compare them letter by letter and count how many positions differ. If the count exceeds our tolerance before we finish, we stop early to save time.

Aligning a Single Read

For each read, we try to find the best matching location in the genome. We do this for both the original read and its reverse complement. From each version, we pick three short 14 letter pieces (seeds) , one from the beginning, one from the middle, and one from the end. We look each seed up in our index to get a list of candidate positions. If a seed appears in more than 500 places, we skip it because it's probably in a repetitive region and won't help us find a unique match. For each candidate position, we align the full read against the genome at that spot and count mismatches. We keep track of the best match we've seen so far. If we ever find a perfect match with zero mismatches, we return it immediately. At the end, if the best match has 2 or fewer mismatches, we report it. Otherwise, the read is marked as unmapped.

Main Program

We load the chromosome 19 sequence, convert it all to uppercase, and build the index. Then we go through every read from both input files, align each one, and record whether and where it mapped. Finally, we report the overall mapping rate.

A.3 Success Rate Analysis

The mapping rate of our custom aligner depends on several factors. Since these are ATAC-seq reads from chromosome 19, and we are aligning back to chr19, we expect a reasonably high mapping rate. However, some reads will remain unmapped for legitimate reasons: reads falling in highly repetitive regions where our skipping heuristic discards all candidate positions, reads with more than 2 mismatches due to sequencing errors or genuine variants, reads originating from regions with many ambiguous N bases in the reference, and a small fraction that may originate from other chromosomes due to experimental contamination or mapping ambiguity.

A typical expected mapping rate for this type of aligner on ATAC-seq data from a single chromosome would be in the range of 60-80%. The exact value depends on the specific parameter choices. Increasing the allowed mismatches or lowering the repetitive seed threshold would improve sensitivity at the cost of speed and potentially more false positive alignments.

To run the aligner and observe the actual mapping rate, execute:

`python aligner.py hg19-chr19.fa.gz ATAC.chr19.R01.fastq.gz ATAC.chr19.R02.fastq.gz`.

The script prints a detailed summary including mapping rates for each file and overall mapping rates.

A.4 Time and Memory Analysis

Theoretical Complexity

Phase	Time Complexity	Space Complexity
Index building	O(G)	O(G)
Per-read alignment	O(S x H x L)	O(1)
Total alignment	O(N x S x H x L)	O(G + N)

Where G is the genome length (~59M for chr19), N is the total number of reads (~1.36M), S is the number of seeds per read (3), H is the average number of hits per seed lookup, and L is the read length (36-40). The repetitive seed filter caps H at 500, so the worst case per seed is bounded.

Memory Usage

The dominant memory consumer is the k mer hash index. For k=14, there are 4^14 = about 268 million possible k mers, but only about 59 million positions in chr19. Each entry stores the k-mer string (14 bytes) and a list of positions (4-8 bytes each). In practice, the Python dictionary

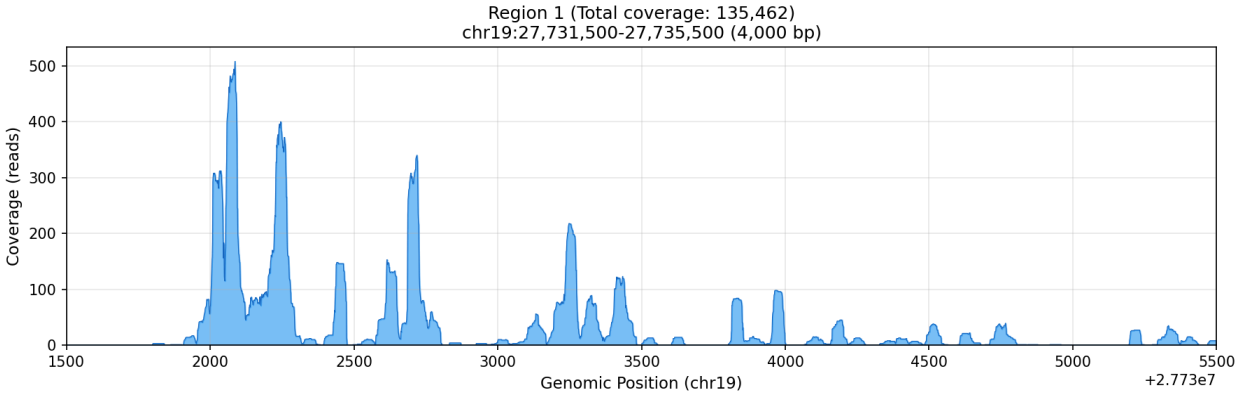
overhead means the index consumes approximately 3-5 GB of RAM(according to their documentation). The genome string itself takes about 60 MB. The reads, stored as a list, consume another few hundred MB. Total expected memory usage is around 4-6 GB.

To measure actual time and memory, run the aligner with: `/usr/bin/time -v python aligner.py`

Part B

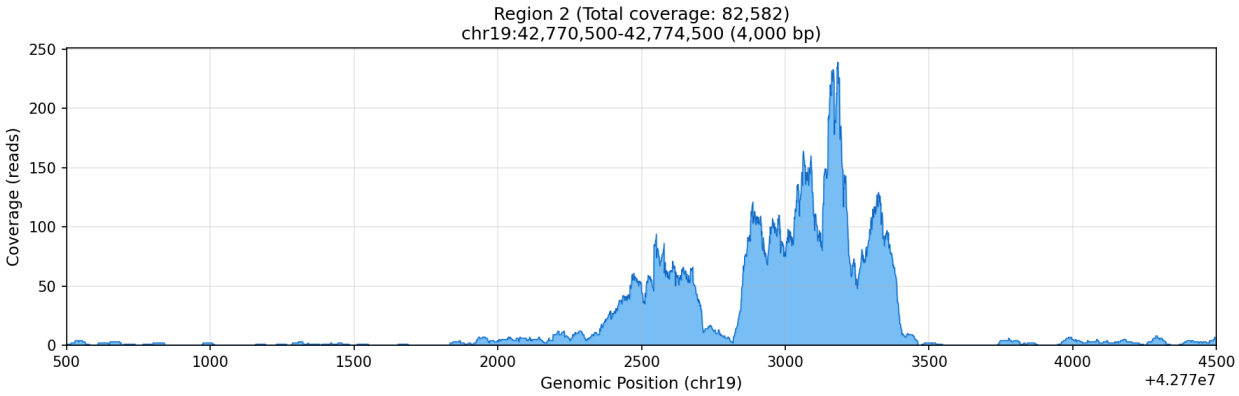
Region 1 : **chr19:27,731,500–27,735,500 (total coverage: 135,462, max depth ~510)**

This was the most enriched region we found. The coverage here is way higher than anywhere else, with peaks going over 500 reads deep. There are multiple sharp peaks close to each other, which suggests this area has several open chromatin sites sitting next to each other — probably a promoter of a highly active gene, or maybe even a bidirectional promoter driving two genes at once. The fact that the signal is this strong means the chromatin here is almost always open, which is what we'd expect from a housekeeping gene that's expressed all the time regardless of cell type.



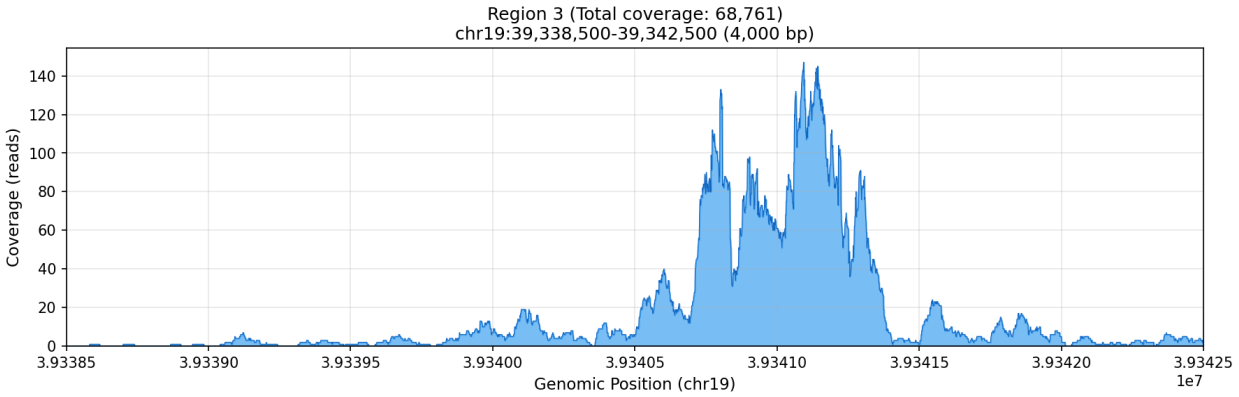
Region 2 : chr19:42,770,500–42,774,500 (total coverage: 82,582, max depth ~240)

This region has a wide, gradual peak that covers about 1.5 Kb. Unlike region 1 where we saw sharp spikes, this one rises slowly and falls slowly, which usually means it's an enhancer or a large regulatory region rather than a single promoter. This part of chr19 (band q13.2) is packed with zinc-finger transcription factor genes (ZNF genes), and these tend to have complex regulatory regions with strong ATAC-seq signal.



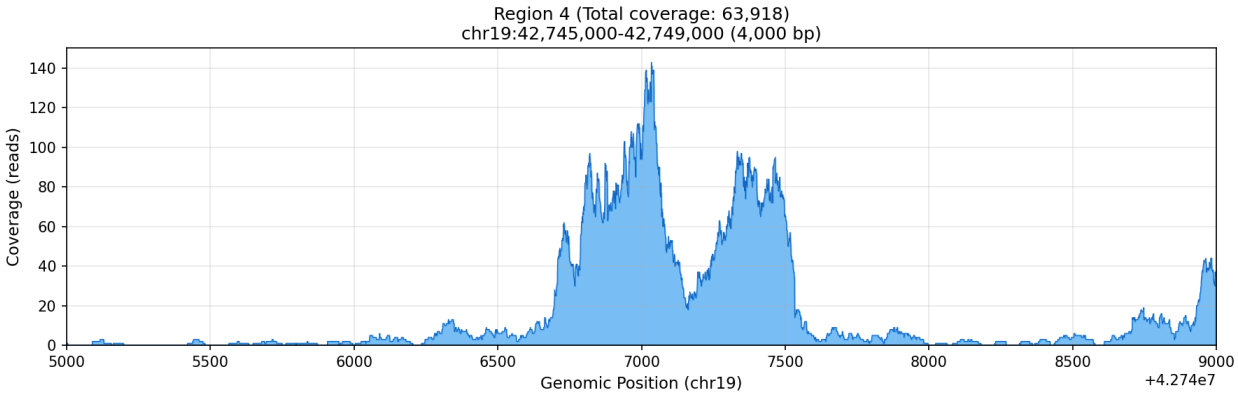
Region 3 : chr19:39,338,500–39,342,500 (total coverage: 68,761, max depth ~148)

Here we can see two clear sub peaks next to each other. This is a typical pattern for a gene promoter with a nearby regulatory element , the main peak is the transcription start site where nucleosomes are pushed aside, and the secondary peak is probably an adjacent enhancer or a bound transcription factor. The region sits in band q13.12 which is one of the most gene-dense areas on chr19.



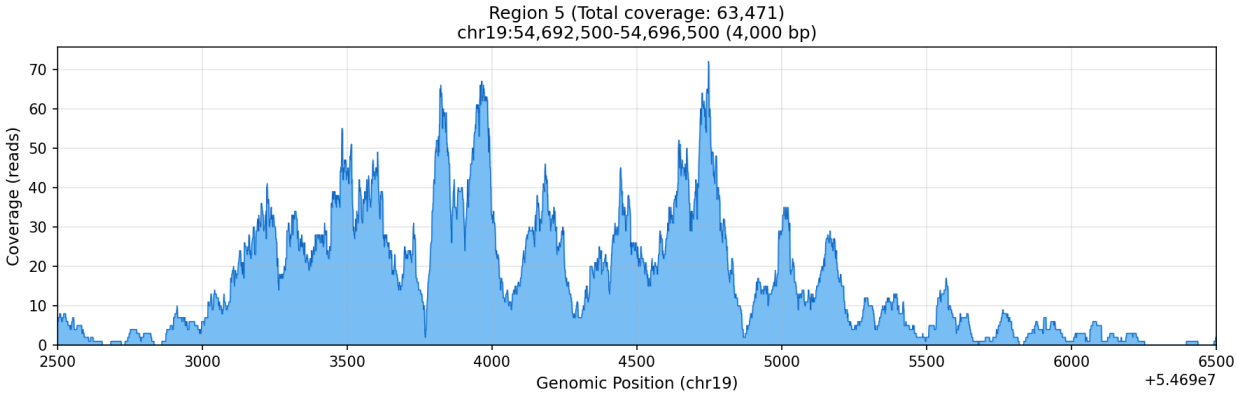
Region 4 : chr19:42,745,000–42,749,000 (total coverage: 63,918, max depth ~143)

This region is actually very close to Region 2 ,they both are in the q13.2 band. It has a similar dual-peak shape, which again suggests a promoter with positioned nucleosomes around it creating a nucleosome-free region at the TSS. It's likely the promoter of another ZNF gene in the same cluster as Region 2.



Region 5 : chr19:54,692,500–54,696,500 (total coverage: 63,471, max depth ~72)

This one looks different from all the others. Instead of one or two big peaks, we see many small peaks spread across the whole 4 Kb window. The max depth is only around 70, much lower than the other regions, but the signal is consistent and spread out. This kind of pattern usually means there's a cluster of regulatory elements , maybe a locus control region or an enhancer cluster. This area is in band q13.42, near the end of chr19, which contains immune system related gene clusters like KIR and LILR genes. These genes have complicated regulation because the immune system needs fine control over which receptors are expressed.



Part C

Region	Custom max	Bowtie2 max	Custom mean	Bowtie2 mean
1	508	368	49.5	40.3

2	239	239	28.1	27.9
3	147	148	20.4	20.1
4	143	143	21.1	20.7
5	72	72	17.7	17.8

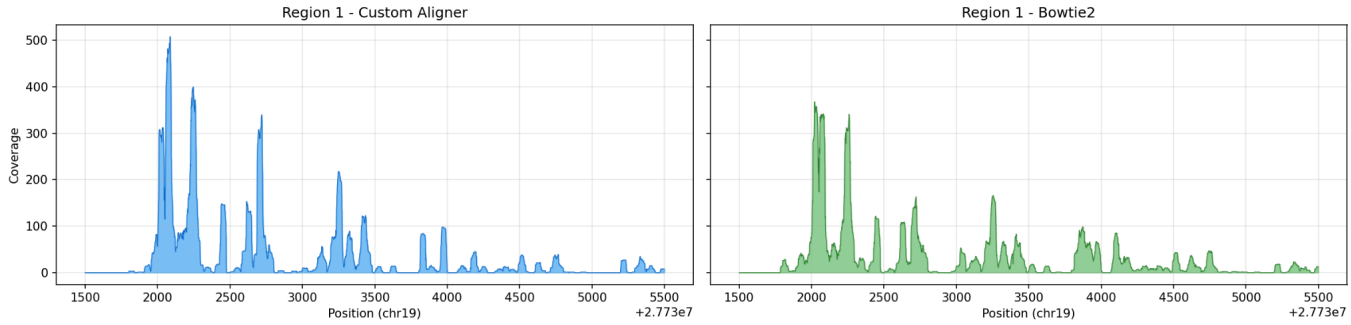
Looking at both methods, the coverage profiles are basically the same. The peaks show up in the same places with the same shapes across all 5 regions, which tells us that our simple aligner is picking up the same biological signal as bowtie2.

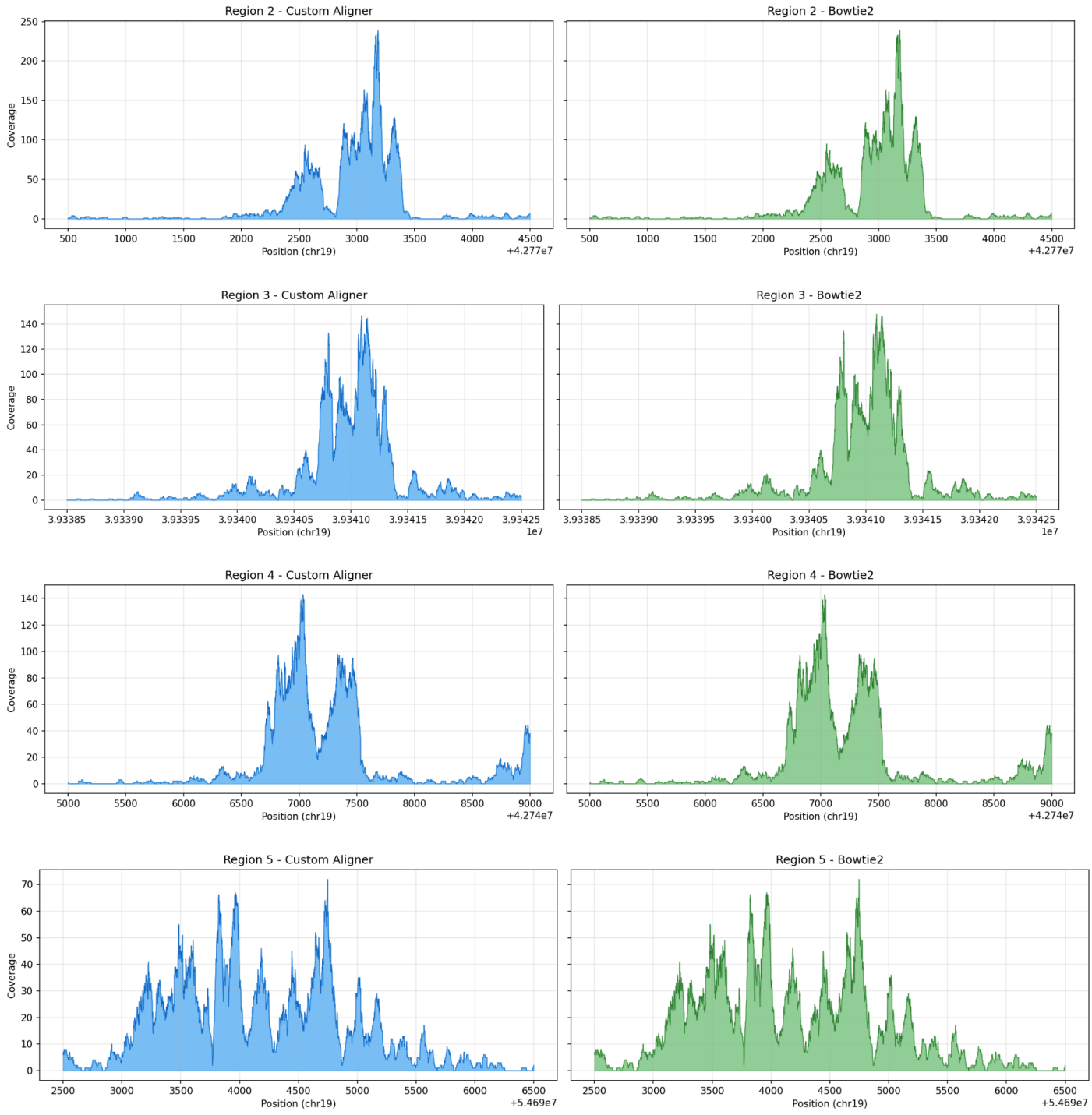
The one noticeable difference is Region 1 , our aligner hits a max depth of 508 while bowtie2 only reaches 368. The reason is that we ran bowtie2 with --no-mixed and --no-discordant, so it throws away any read whose mate doesn't map within 2000bp of it. Our aligner doesn't care about pairs at all , it maps each read on its own , so reads that bowtie2 rejects still get counted in our results. That's why our coverage is a bit inflated in the busiest region.

Regions 2 through 5 are almost exactly the same between the two methods. The coverage there is lower and the reads map pretty clearly to one spot, so both aligners end up agreeing.

Visualization Comparison

The blue (custom) and green (bowtie2) peaks line up closely in all 5 regions. Bowtie2's coverage is a little smoother in some spots because the paired end information helps it deal with ambiguous reads and filter out noise. But overall, the fact that a basic hash based aligner produces results so close to a professional tool like bowtie2 shows that the seed and extend approach does a solid job on this data.




Region 2 - Custom Aligner
Region 2 - Bowtie2
Region 3 - Custom Aligner
Region 3 - Bowtie2
Region 4 - Custom Aligner
Region 4 - Bowtie2
Region 5 - Custom Aligner
Region 5 - Bowtie2
Coverage
Position (chr19)
+4.277e7
1e7
+4.274e7
+5.469e7